

1. General Information

- Title: Map guessing game
- Creator: Minh Tran
- Student number: 101555658
- Study program: Digital Systems and Design
- Study year: third year
- Date: 20/2/2026

2. General Description and Difficulty Level

- This project is an interactive desktop application that teaches world geography through exploration and competitive gaming. The application features an interactive map where users can discover country details, a "Guess by Shape" mode with visual hints, a "Flag Master" mode, and a high-speed "Time Attack" challenge.
- Unique feature:
 - o A "Dynamic Heatmap" progress system. As the user correctly identifies countries, the world map is permanently updated with a color gradient representing the user's mastery level of different continents.
 - o A counter to track the distance traveled after correctly guessing a country (distance between capitals). Using Haversine formula to calculate distance on a sphere.
- Difficulty level: I'm aiming for the hard level.

3. Use Case and User Interface

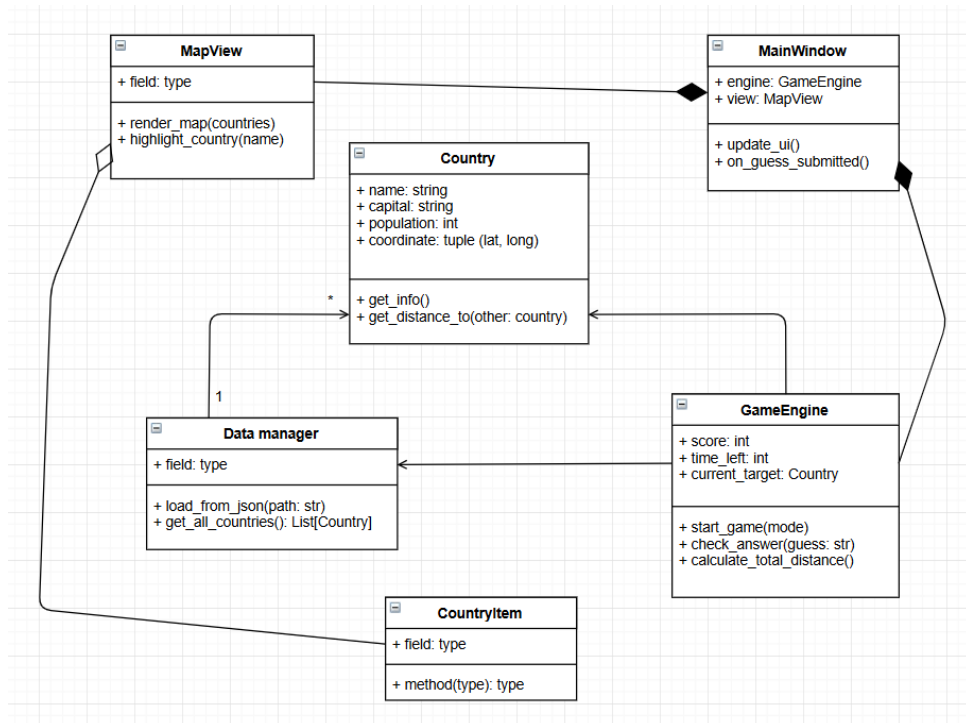
- UI description: the main window consists of a large QGraphicsView displaying a world map. On the right, a side panel shows statistics (score, progress). At the top, a navigation bar allows switching between "Explore," "Shape Quiz," "Flag Quiz," and "Time Attack."
- Use case:
 - o User action: click the "Time Attack" and "Start"
 - o Behind the scene: the GameEngine initializes a QTimer (60s) and selects a random country
 - o User action: sees a country outline (shape quiz) and types the name into a QLineEdit
 - o Behind the scene: GameEngine compares input to the Country object's name. If correct, it increments the score and triggers a

"Success" signal to the MapView to highlight the country in green.
Also provide some information about the country.

- User action: if feeling stuck, clicks "Hint"
 - Behind the scene: the program reveals some hints (like the capital, etc)

4. Structure of the Program

- The program follows a Model-View-Controller (MVC) inspired architecture to separate data from the GUI.
 - Country (model): Stores attributes (name, capital, population, flag_path, coordinates, SVG path).
 - DataManager: Read JSON files and turn that into a list of Country objects.
 - GameEngine: Manages game logic, scoring, timers, and hint generation.
 - MainWindow and MapView (UI): This is PyQt side:
 - MainWindow hold buttons and labels
 - MapView (extending QGraphicsView) draws the CountryItem shapes
 - CountryItem: A custom QGraphicsPathItem representing a single country on the map. It knows when it is clicked.



5. Data Structures

- List of Country objects: For sequential access during the quiz
- Dictionary (Hash map): To map country names to their corresponding Country objects for O(1) lookup speed when the user types a guess
- Set: to track “Already Guessed” countries to ensure no duplicates in a single session.
- QPainterPath: To store the complex geometric shapes of country borders for rendering.

6. Files and File Formats

- countries.json: a text file containing all country data (name, capital, etc)
- map_data.svg: containing the coordinates for country outlines
- /flags/ folder: a directory of files named by country code
- userdata.json: stores the user’s progress and high scores

7. Algorithms

- String matching: use a basic fuzzy matching approach
- Randomization: ensure quiz order is random and unique every time
- Coordinate mapping: a linear transformation algorithm to map geographic coordinates to the x, y coordinate system of the QGraphicsScene.
 - $x = (\text{longitude} + 180) \times (\text{width} / 360)$
 - $y = (90 - \text{latitude}) \times (\text{height} / 180)$
- Logic for “hint”: a queue of hint prioritized.

8. Testing Plan

- Unit testing:
 - Test `GameEngine.check_guess()`
 - Test `DataManager.load_data()` to prevent the program crash if JSON failed
- UI and system testing:
 - Verify clicking a country open the correct info panel
 - Test Qtime stops exactly a zero
- Boundary test:

- Test countries with special name (2 word name, contain special character, etc)

9. Libraries and Tools

- PyQt6: for graphical UI and event handling
- json: for data storage
- random: for random quiz

10. Schedule

- Research and data preparation (5h): JSON and SVG data gathered
- Code architecture (8h): Data classes and basic QGraphicsView map rendering.
- Checkpoint 3 (10h): Working map when clicking a country show its name
- Game logic (10h): Implementation of Shape Quiz and Flag Quiz modes
- Advanced features (8h): Timer challenge, scoring system, and hints.
- Checkpoint 6 (6h): Unit tests for game logic and data loading completed
- Polishing and UI (5h): Final visual tweaks and bug fixing.

11. References

- PyQt6: <https://www.riverbankcomputing.com/static/Docs/PyQt6/>
- RestCountries API (for data source): <https://restcountries.com/>
- Natural earth (for map vector data): <https://www.naturalearthdata.com/>
- Python 3.x standard library documentation